

CSF2753: Web Service Development

Filippos Pantekis

September 23, 2026

Lab Class 1 - Distributed Programming with Java

In this lab we will be exploring basic distributed programming with the `java.rmi` and `java.net` packages. You may find the following trails, provided by Oracle, useful:

<https://docs.oracle.com/javase/tutorial/networking/sockets/index.html>

<https://docs.oracle.com/javase/tutorial/rmi/index.html>

I recommend creating a directory / folder for the module labs. In addition, all code used for lecture demonstrations is available on Canvas, under:

Modules → Week_1 → Lab

You may use these files as reference points for your own implementation, copy and edit them or ignore them entirely. I do not recommend the latter option.

If you have built your own Java project within your respective IDE, you will need to move the git code into the `src` folder in order to run it.

To receive a sign-off for this module's tasks, you should be able to explain how you arrived at the code you are showing. Be prepared to answer questions.

Tasks continue on the following page.

1 Client-Server Socket Programming [1 Mark]

Task 1.1. Create a java program implementing a UDP interaction between a *client* and *server*. The *server*, upon receiving a 'connection' **without** any request data, should send a random number to the *client*.

You may find the following API reference useful, in terms of explaining the methods and parameters.

<https://docs.oracle.com/javase/8/docs/api/java/net/DatagramSocket.html>

<https://docs.oracle.com/javase/8/docs/api/java/net/DatagramPacket.html>

Task 1.2. Create a java program implementing a TCP connection between a *client* and *server*. The *server* should receive **two** numbers from the *client*, and return the sum (addition) of those two numbers to the *client*.

You may find the following API reference useful, in terms of explaining the methods and parameters.

<https://docs.oracle.com/javase/8/docs/api/java/net/Socket.html>

<https://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html>

2 Extending Functionality [1 Mark]

Task 2.1. Extend your TCP implementation of both *client* and *server* by turning it into a basic calculator program.

The *client* should be able to send at least **two** numbers and request an **operation** to be performed on those two numbers. At minimum, the server should be able to support:

- addition (add)
 - subtraction (sub)
 - multiplication (mul)
 - division (div)
-

3 Game of Challenges [1 Mark]

Filippos runs a simple TCP server that supplies text challenges. The server has IP (domain): **nullpointer.click** and runs on port **9293**. When a client connects to the server, the server sends 2 lines of text introducing the user to the game. After that, the challenge game starts, and the server:

- Sends a line of text explaining the challenge.
- Awaits for a single line of text containing the answer
- Sends a line of text with the outcome of the challenge (right/wrong).

The above 3 steps continue until the wrong answer is sent to one of the challenges.

Task 3.1. Create a Java program to interact with the quiz server. Your program must show the initial messages to the user, then start an infinite loop that shows the challenge, reads the user's answer from the keyboard and sends it to the server, and reads and prints the outcome of the challenge. If the server indicates the result was wrong, you should end the program and loop.

Notes:

- To open a socket to the server: `new Socket(InetAddress.getByName("nullpointer.click"), 9293)`
 - Your code must follow the above protocol exactly, otherwise you will fall "out of sync" with the server!
 - The server running at **nullpointer.click:9293** is going to be active for two weeks after the release of this task. Subsequently, the code for the server will be released so you can run it locally.
 - This is one of the rare occasions where an infinite loop and use of 'break' are allowed.
-

4 Challenge: Java RMI Application

Task 4.1. *Implement the addition program from Task 1.2 as an RMI Application.*

Remember that demonstration code can be found on Canvas. I would suggest you start with defining an **Interface**. If you want to test your understanding further, extend the program functionality to become a basic calculator.